

GPU 模拟器自动校准框架

面向 AI 模型的三机制诊断方法

汇报人：XXX

日期：2026-04-11

问题：模拟器校准的根本困难

现有 DSE 框架（包括 ChronosDSE）有一个隐含前提：

■ 模拟器本身是准确的。

但没有人系统性地回答：

- **校准哪些 kernel?** 对所有 kernel 全量校准代价太高
- **软件层干净吗?** 软件缺陷会污染架构信号

未校准的模拟器 → 时间窗口分析 → DSE 结论

↑
可靠性存疑

我们的目标

第一阶段（我们的工作）

真实硬件 NCU 数据



软件层清洗



筛选最小 kernel 子集



模拟器精确校准

第二阶段（ChronosDSE 的方法）

校准好的模拟器



cycle 级时间窗口分析



发现 kernel 内部动态性



—→ 尾部感知 DSE（可信）

我们的工作是 ChronosDSE 的前置保障层。

方法论：三阶段诊断协议

Stage A: 启动配置检查

`waves_per_sm / occupancy` 是否合理?

→ 不通过: 修复 `launch` 配置 (软件问题)

↓ 通过

Stage B: Kernel 实现检查

`warp_cycles / compute` 是否合理?

→ 不通过: 修复 `kernel` 实现 (软件问题)

↓ 通过

Stage C: 架构瓶颈诊断

软件层已干净, 信号来自真实硬件架构

→ 生成模拟器校准处方

核心思想: 软件问题必须先于架构分析被清除。

三机制压缩框架

在 Stage C 之前，用三个机制对 kernel 数据做结构化压缩：

机制	维度	问题	输出
Squash	时间	算法有几种硬件工作状态？	段结构 + 凝聚度
Batch	空间	哪些 kernel 是异类？	聚类 + outlier
Delta	差异	什么在驱动 kernel 间的差异？	相关性 + 中心字段

目标：把 AI 诊断的输入从原始数据压缩为高信噪比的结构化摘要。

关键实验：mini-transformer 软件清洗

二进制演进过程：

版本	修改	warp_cycles	compute%	状态
v1	原始	174.7	22.4%	Stage B 不通过
v2	4-wide 累加器 (B-1)	169.7	22.4%	无效
v3	shared memory 预加载	121.6	17.1%	bank conflict
v4	shmem + padding (B-4)	34.0	95.2%	 通过

B-1 误诊的原因： 编译器 (-O2) 已自动生成 ≥ 8 -wide 累加器，真正瓶颈是 LDG 加载延迟 (28 cycles) ，需要 shared memory 预加载。

软件清洗对架构信号的影响

Delta 在 v1 vs v4 上的相关性对比：

信号	v1（软件有缺陷）	v4（软件清洗后）	解读
l1_hit ↔ compute	-0.646	≈ 0（消失）	v1 信号是软件缺陷投影
block_limit_registers ↔ compute	弱/噪声	-0.946	清洗后才可见
block_limit_registers ↔ dram	弱/噪声	+0.957	清洗后才可见

结论： 没有三阶段协议的软件清洗，
Delta 产出的是软件噪声而非架构信号。

Squash + Batch: 解决长尾 kernel 问题

mini-transformer v4 的 Batch 输出:

聚类 A: gemm_tiled ×7	同质度 1.0	→ 1 个代表
聚类 B: residual_add ×2	同质度 1.0	→ 1 个代表
聚类 C: layernorm ×2	同质度 1.0	→ 1 个代表

Outlier: attention_score / softmax / context_mul → 各需单独模拟

效果: 14 次 launch → 6 个代表, 模拟次数压缩 **57%**

解决长尾问题:

- 传统做法: 按运行时间排序, 只模拟头部 kernel
- 我们的做法: 按行为差异聚类, **outlier 无论运行时间长短都被保留**
- residual_add 运行时间短, 但是唯一的 HBM 流式 kernel, 是 HBM 带宽校准的唯一基准

Stage C 架构处方 (v4)

Delta 发现 `block_limit_registers` 是中心约束:

```
block_limit_registers ↔ compute_throughput = -0.946
block_limit_registers ↔ l1_throughput      = -0.964
block_limit_registers ↔ dram_throughput     = +0.957
```

生成的模拟器校准处方:

处方	Kernel	目标参数	置信度
C-1	gemm + attention	<code>gpgpu_shader_registers</code>	HIGH
C-2	residual_add	<code>gpgpu_n_mem</code> , HBM 时序	HIGH
C-3	softmax	<code>gpgpu_cache:d12</code>	MEDIUM

跨 Workload 验证结果

Workload	处方	验证方式	结果
backprop	DP initiation 16→4	模拟器实测	IPC +77% 
backprop	shmem banks 32→64	模拟器实测	无效（模拟器不建模）
mini-transformer	softmax 启动修复（A-1）	硬件实测	occupancy +77pp 
mini-transformer	attention B-4 修复	硬件实测	warp_cycles -80.5% 

置信度标注可靠：HIGH 处方全部生效，MEDIUM 处方需要进一步验证。

与 ChronosDSE 的关系

ChronosDSE 的两个隐含前提：

前提	ChronosDSE	我们的框架
知道分析哪些 kernel	✗ 全量分析	✓ Squash+Batch 筛选最小子集
软件层是干净的	✗ 未处理	✓ 三阶段协议保证

两个工作的分工：

我们的框架		ChronosDSE
校准模拟器（保证可信）	→	在可信模拟器上做尾部分析
筛选最小 kernel 子集	→	对子集做 cycle 级动态性分析
软件层清洗	→	专注架构层动态性

一句话：我们的工作让 ChronosDSE 的结论变得可信。

下一步计划

第一优先级：完成模拟器校准闭环

- 在模拟器上验证 Stage C 处方 (C-1/C-2/C-3)
- 量化校准前后模拟器预测误差的变化

第二优先级：实现第二阶段

- 在校准好的模拟器上，对最小 kernel 子集做 cycle 级时间窗口分析
- 验证能否复现 ChronosDSE 发现的瓶颈迁移现象

第三优先级：扩展到大模型

- 在 LLaMA / GPT-2 上验证框架的泛化性
- 量化两阶段框架相比全量分析的效率提升

总结

已完成：

- 三阶段诊断协议（含 B-4 新判据）
- 三机制压缩框架（Squash / Batch / Delta）
- 3 个 workload 的完整消融实验（E0-E4）
- 关键处方的硬件/模拟器闭环验证

核心贡献：

第一个系统性解决"模拟器校准用哪些 kernel、软件层是否干净"的框架，
为 ChronosDSE 类工作提供可信的模拟器基础。

目标会议： MICRO / ISCA